TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1 · HỌC KỲ 1, NĂM HỌC 2026 – 2027

Tài liệu tự học buổi 13 — Kiểm thử theo chuyên ngành

Công cụ kiểm thử và kiểm thử API [SE] · Ba chiều chất lượng dữ liệu: đầy đủ, chính xác, nhất quán [DA] · Kiểm định chéo, confusion matrix và đo thiên lệch [AI]. Mỗi phần có mã mẫu chạy được và mẫu Test Report điền sẵn một dòng.

Sản phẩm của tài liệu này là thành phần 1, 2 và 3 của Bài tập 3 — Test Plan, Test Report và Bug Log. Làm ở workshop buổi 14, nộp cuối buổi 16.

Biên soạn: **TS. Nguyễn Trí Hải** · Bộ môn Kỹ thuật phần mềm

Tài liệu lưu hành nội bộ · Bộ tài liệu tự học phần chuyên ngành (30% thời lượng lý thuyết buổi 13). Tài liệu song ngữ: bản tiếng Việt ở nửa đầu, bản tiếng Anh (English version) ở nửa sau.

Cách dùng tài liệu này

Giảng viên giảng trên lớp (buổi 13)	Sinh viên tự đọc trong tài liệu này
Vì sao "em đã thử rồi" không phải bằng chứng; bốn mục của Test Plan rút gọn; tám cột bắt buộc của một test case; ba nguồn để chọn test case; giá trị biên; bảy dòng của một mục Bug Log; bốn bước gỡ lỗi; mức độ bug.	Công cụ và kỹ thuật cụ thể của track mình: chạy kiểm thử bằng công cụ nào và viết như thế nào [SE]; ba chiều chất lượng dữ liệu và cách đo từng chiều [DA]; kiểm định chéo, đọc confusion matrix và đo thiên lệch giữa các nhóm [AI].

Bốn điều đúng cho cả ba track — đọc trước khi vào phần của mình

1. MỘT TEST CASE PHẢI TÁI LẬP ĐƯỢC BỞI NGƯỜI KHÁC. Nghĩa là nó phải ghi rõ tiền điều kiện, dữ liệu thử cụ thể, và kết quả mong đợi cụ thể. "Kiểm tra chức năng tra cứu hoạt động đúng" không phải test case — đó là một mong muốn.

2. GHI KẾT QUẢ TRUNG THỰC. Giảng viên sẽ chọn ngẫu nhiên một test case em ghi PASS và yêu cầu em chạy lại tại chỗ ở buổi 14. Ghi FAIL trung thực rồi sửa là cách được điểm cao nhất; ghi PASS khống là vấn đề liêm chính.

3. KẾT QUẢ "GẦN ĐÚNG" LÀ FAIL. Nếu test mong đợi mã 409 mà hệ thống trả 400, đó là FAIL, dù cả hai đều là lỗi. Du di ở đây là chỗ sinh viên mất điểm nhiều nhất.

4. GẶP BUG THÌ GHI BUG LOG NGAY, KHÔNG ĐỂ DỒN. Nguyên nhân của một bug chỉ còn rõ ràng trong khoảng mười phút sau khi em gỡ xong. Để tới cuối buổi thì em chỉ còn nhớ "sửa gì đó ở hàm nào đó" và Bug Log sẽ sơ sài.

Nhắc lại: tám cột bắt buộc của một test case và bảy dòng của một mục Bug Log

Tám cột của test case (đề BT3, mục 2.2)	Bảy dòng của một mục Bug Log
1. Mã test case (TC-01, TC-02…) 2. Tiền điều kiện — trạng thái dữ liệu và hệ thống trước khi chạy 3. Các bước thực hiện — đánh số, mỗi bước một hành động 4. Dữ liệu thử — giá trị CỤ THỂ, không phải "một số hợp lệ" 5. Kết quả mong đợi — quan sát được, đo được 6. Mức ưu tiên — Cao / Trung bình / Thấp Hai cột cuối điền khi CHẠY THẬT: 7. Kết quả thực tế — bằng lời cụ thể 8. PASS / FAIL	1. Mã bug (BUG-01…) và mức độ (Critical / Major / Minor) 2. Phát hiện bởi test case nào 3. Triệu chứng — người dùng thấy gì 4. Cách tái lập — các bước tối thiểu để bug xuất hiện lại 5. Nguyên nhân — nguyên nhân THẬT, không phải triệu chứng 6. Cách sửa — sửa gì, ở tệp nào 7. Link commit sửa bug + xác nhận đã chạy lại test case và PASS

PHẦN A — Track SE: Công cụ kiểm thử và kiểm thử API

A1. Ba tầng kiểm thử và tỉ lệ hợp lý cho một prototype

Tầng	Kiểm cái gì	Số lượng hợp lý ở Bài tập 3	Chi phí viết và bảo trì
Unit test	Một hàm hoặc một lớp, tách khỏi mọi phụ thuộc bên ngoài (dùng đối tượng giả)	6 – 8 test, tập trung vào lớp service	Rẻ nhất, chạy nhanh nhất (vài mili giây), ổn định nhất
Integration test	Nhiều thành phần thật làm việc cùng nhau, thường gồm cơ sở dữ liệu thật	2 – 3 test cho các luồng quan trọng nhất	Trung bình; cần dựng và xoá dữ liệu thử
API / End-to-end test	Toàn hệ thống qua giao diện ngoài (HTTP), như người dùng thật	2 – 3 test cho luồng nghiệp vụ chính	Đắt nhất, chậm nhất, dễ vỡ nhất khi hệ thống đổi

Vì sao tỉ lệ này chứ không phải tỉ lệ ngược lại

Unit test rẻ nên viết được nhiều; nó bắt được phần lớn lỗi logic nghiệp vụ và chỉ ra ngay hàm nào sai.

Kiểm thử đầu-cuối đắt nên viết ít; nó bắt được lỗi tích hợp mà unit test không thấy, nhưng khi nó đỏ thì em vẫn phải đi tìm nguyên nhân ở đâu.

Sai lầm phổ biến: chỉ viết kiểm thử giao diện. Chúng chậm, hay vỡ vì lý do không liên quan, và khi vỡ thì không nói được lỗi nằm ở đâu.

Với Bài tập 3, 10–12 test case theo tỉ lệ trên là đủ. Con số quan trọng hơn là: có ít nhất một test cho MỖI quy tắc nghiệp vụ em đã hiện thực.

A2. Công cụ theo stack — chọn một và ghi lệnh chạy vào README

Stack	Công cụ đề xuất	Lệnh chạy	Ghi chú
Node.js / Express	Jest hoặc Vitest, kèm Supertest cho kiểm thử API	npm test	Supertest gọi được ứng dụng Express mà không cần mở cổng thật — nhanh và ổn định hơn gọi qua HTTP.
Java / Spring Boot	JUnit 5 + Mockito, kèm MockMvc hoặc @SpringBootTest	mvn test hoặc ./gradlew test	MockMvc kiểm thử tầng web mà không khởi động máy chủ đầy đủ.
Python / FastAPI, Flask, Django	pytest, kèm TestClient của FastAPI hoặc test client của Django	pytest -q	pytest fixture là công cụ tốt nhất để dựng và dọn dữ liệu thử.
Kiểm thử API thủ công (mọi stack)	Postman collection đã xuất tệp, hoặc tệp .http của REST Client	Chạy trong Postman, hoặc newman run <tệp>	Bắt buộc phải xuất tệp và commit vào repo — collection chỉ nằm trên máy em thì không phải minh chứng.

A3. Mã mẫu — ba test cho một quy tắc nghiệp vụ

tests/warranty.test.js — Jest, kiểm quy tắc hạn bảo hành QT-05

```
const TicketService = require('../src/services/ticket.service');

describe('QT-05 xác định còn hạn bảo hành', () => {
  const svc = new TicketService(null, null, null);   // hàm thuần, không cần repo

  // TC-01 LUỒNG ĐÚNG
  test('TC-01 thiết bị bán cách đây 3 tháng, bảo hành 12 tháng -> còn hạn', () => {
    expect(svc.isUnderWarranty('2026-06-01', 12, new Date('2026-09-01'))).toBe(true);
  });

  // TC-02 GIÁ TRỊ BIÊN - hai test ở đúng hai bên ranh giới
  test.each([
    ['2027-03-01', true,  'đúng ngày hết hạn -> VẪN còn bảo hành'],
    ['2027-03-02', false, 'sau ngày hết hạn một ngày -> HẾT bảo hành'],
  ])('TC-02 biên: %s -> %s (%s)', (ngayKiem, kyVong) => {
    expect(svc.isUnderWarranty('2026-03-01', 12, new Date(ngayKiem))).toBe(kyVong);
  });

  // TC-03 DỮ LIỆU SAI
  test('TC-03 ngày bán không đọc được -> ném lỗi, KHÔNG trả về true', () => {
    expect(() => svc.isUnderWarranty('khong-phai-ngay', 12, new Date()))
      .toThrow();
  });
});
```

tests/tickets.api.test.js — Supertest, kiểm thử API qua HTTP

```
const request = require('supertest');
const { buildApp } = require('../src/app');
const { resetDb, seedDemo } = require('./helpers/db');

describe('POST /api/tickets', () => {
  let app;
  beforeEach(async () => {
    await resetDb();          // TIỀN ĐIỀU KIỆN được dựng bằng code,
    await seedDemo();         // không phụ thuộc dữ liệu có sẵn trên máy
    app = buildApp();
  });

  test('TC-04 tạo phiếu thành công trả về 201 và có ticket_code', async () => {
    const res = await request(app).post('/api/tickets').send({
      customerId: 1, serialNo: 'SN10293A', productId: 5, centerId: 2,
      categoryId: 1, priority: 'CAO',
    });
    expect(res.status).toBe(201);
    expect(res.body.data.ticket_code).toMatch(/^BH-\d{6}\/\d{4}$/);   // vd BH-000231/2026
  });

  test('TC-05 thiếu trường bắt buộc -> 400 và thông báo NÊU RÕ trường nào', async () => {
    const res = await request(app).post('/api/tickets').send({ customerId: 1 });
    expect(res.status).toBe(400);
    expect(res.body.message).toMatch(/serialNo/);      // thông báo phải nêu tên trường
  });

  test('TC-06 serial đã có phiếu đang mở -> 409, KHÔNG phải 400 hay 500 (QT-03)', async () => {
    await request(app).post('/api/tickets').send({
      customerId: 1, serialNo: 'SN10293A', productId: 5, centerId: 2,
      categoryId: 1, priority: 'CAO' });
    const res = await request(app).post('/api/tickets').send({
      customerId: 1, serialNo: 'SN10293A', productId: 5, centerId: 2,
      categoryId: 1, priority: 'CAO' });
    expect(res.status).toBe(409);
    expect(res.body.error).toBe('PHIEU_DANG_MO');
  });
});
```

Ba điều đoạn mã trên minh hoạ — sẽ được hỏi ở buổi 14

1. Tiền điều kiện được dựng bằng code, không giả định. resetDb() và seedDemo() bảo đảm test chạy được trên máy bất kỳ. Test phụ thuộc dữ liệu có sẵn trên máy em thì không phải minh chứng.

2. Kiểm cả MÃ TRẠNG THÁI và NỘI DUNG. TC-05 không chỉ kiểm 400 mà kiểm cả thông báo có nêu tên trường thiếu. Một thông báo lỗi vô dụng vẫn trả về 400.

3. Phân biệt được 400, 409 và 500. 400 là sai định dạng · 409 là xung đột trạng thái nghiệp vụ · 500 là lỗi không lường trước. Một hệ thống trả 500 cho lỗi nghiệp vụ đã biết là thiết kế sai xử lý lỗi.

A4. Kiểm thử bảo mật ở mức nhận biết — năm mục ưu tiên từ OWASP Top 10:2025

Ở buổi 13 em chỉ cần KIỂM ĐƯỢC năm mục dưới đây bằng tay và ghi kết quả. Rà soát bảo mật đầy đủ thuộc phạm vi Chuyên đề tốt nghiệp 2. Danh mục dưới đây theo bản OWASP Top 10:2025.

Mục OWASP	Nghĩa là gì	Cách em tự kiểm bằng tay	Kết quả mong đợi
A01:2025 Broken Access Control	Người dùng truy cập được thứ không thuộc quyền của mình	Đăng nhập bằng tài khoản A, lấy id một bản ghi của tài khoản B, gọi API sửa hoặc xoá bản ghi đó	403 hoặc 404, KHÔNG được là 200. Đây là mục quan trọng nhất và bị bỏ sót nhiều nhất.
A02:2025 Security Misconfiguration	Cấu hình để lộ thông tin hoặc mở quá rộng	Gọi một endpoint gây lỗi và đọc phản hồi; kiểm CORS có mở cho mọi nguồn không	Phản hồi lỗi KHÔNG chứa vết ngăn xếp, tên bảng, hay chuỗi kết nối. CORS chỉ mở cho nguồn cần thiết.
A04:2025 Cryptographic Failures	Dữ liệu nhạy cảm không được bảo vệ đúng cách	Truy vấn trực tiếp bảng người dùng và xem cột mật khẩu	Phải là chuỗi băm (bcrypt, argon2), tuyệt đối không phải mật khẩu đọc được.
A05:2025 Injection	Dữ liệu người dùng bị hiểu thành mã lệnh	Nhập vào một trường tìm kiếm chuỗi: ' OR 1=1 -- và một chuỗi thẻ script	Không trả về toàn bộ bản ghi; chuỗi thẻ hiển thị dạng văn bản chứ không được thực thi.
A07:2025 Authentication Failures	Cơ chế xác thực có lỗ hổng	Gọi một endpoint cần đăng nhập mà không gửi token; gửi token đã hết hạn hoặc token bị sửa một ký tự	Cả ba trường hợp phải trả 401. Token sửa một ký tự vẫn dùng được là lỗi nghiêm trọng.

Cách ghi kết quả kiểm bảo mật vào Test Report

Mỗi mục là một test case đủ tám cột, như mọi test case khác. Không được ghi kiểu "đã kiểm bảo mật, đạt".

Ví dụ đúng: TC-09 · Tiền điều kiện: có hai tài khoản A và B, B có phiếu id=7 · Bước: đăng nhập A, gọi PATCH /api/tickets/7 · Dữ liệu thử: token của A, id 7 · Kết quả mong đợi: 403 · Kết quả thực tế: 403, thông báo "Không có quyền" · PASS.

Nếu em phát hiện lỗ hổng thật: ghi ngay một mục Bug Log mức Critical, sửa, và ghi link commit. Phát hiện được lỗ hổng của chính mình là điểm cộng, không phải điểm trừ.

Nếu một mục không áp dụng: ghi rõ "không áp dụng" kèm lý do một dòng, ví dụ "hệ thống chưa có cơ chế đăng nhập nên A07 không áp dụng ở phiên bản này".

A5. Checklist tự kiểm trước khi tới workshop buổi 14 [SE]

- ☐ Có công cụ kiểm thử đã cấu hình và một lệnh duy nhất chạy được toàn bộ, đã ghi vào README.
- ☐ Có ít nhất một test cho MỖI quy tắc nghiệp vụ đã hiện thực.
- ☐ Có ít nhất hai test giá trị biên (hai bên của một ranh giới).
- ☐ Có ít nhất hai test dữ liệu sai, kiểm cả mã trạng thái và nội dung thông báo.
- ☐ Test dựng tiền điều kiện bằng code, không phụ thuộc dữ liệu có sẵn trên máy.
- ☐ Có Postman collection hoặc tệp .http đã xuất và commit vào repo.
- ☐ Đã tự kiểm năm mục OWASP ở mục A4 và ghi kết quả từng mục.
- ☐ Phản hồi lỗi của hệ thống không để lộ vết ngăn xếp, tên bảng hay chuỗi kết nối.
- ☐ Mật khẩu trong cơ sở dữ liệu là chuỗi băm, không đọc được.
- ☐ Đã có ít nhất một mục Bug Log thật, với link commit mở được.

PHẦN B — Track DA: Ba chiều chất lượng dữ liệu

B1. Ba chiều chất lượng — định nghĩa và cách đo

Với track DA, "kiểm thử" nghĩa là đo chất lượng dữ liệu đầu ra của pipeline, chứ không chỉ kiểm mã nguồn. Ba chiều dưới đây là ba chiều bắt buộc phải đo trong Bài tập 3.

Chiều	Câu hỏi nó trả lời	Công thức đo	Ngưỡng gợi ý cho bộ dữ liệu Mekong Mobile
ĐẦY ĐỦ (completeness)	Có bao nhiêu phần dữ liệu bị thiếu, và thiếu ở đâu?	Với mỗi cột: số dòng có giá trị / tổng số dòng. Đo THEO CỘT, không đo trung bình toàn bảng.	so_dien_thoai ≥ 90% (dữ liệu gốc thiếu ~6,1%) · email không đặt ngưỡng (thiếu nhiều là bình thường) · mọi khoá và mọi cột ngày ≥ 99,9%
CHÍNH XÁC (accuracy)	Dữ liệu có khớp với thực tế hoặc với quy tắc đã biết không?	Số dòng thoả quy tắc / tổng số dòng kiểm. Cần một quy tắc hoặc một nguồn đối chiếu.	thanh_tien = so_luong × don_gia ≥ 98% (dữ liệu gốc lệch ~1,5%) · số điện thoại chuẩn hoá được ≥ 90% · closed_at ≥ received_at = 100%
NHẤT QUÁN (consistency)	Cùng một thực thể có được biểu diễn giống nhau ở mọi nơi không?	Số cặp/nhóm nhất quán / tổng số nhóm kiểm. Thường kiểm giữa hai bảng hoặc trong một cột phân loại.	Mọi order_id trong order_items tồn tại trong orders = 100% · cặp (ma_cua_hang, ma_don) duy nhất = 100% · giá trị priority chỉ thuộc ba giá trị cho phép = 100%

Vì sao phải đo ĐẦY ĐỦ theo từng cột, không đo trung bình toàn bảng

Một bảng có 20 cột, 19 cột đầy 100% và cột khoá ngoại thiếu 30% → trung bình toàn bảng là 98,5%, nghe rất tốt. Nhưng cột thiếu 30% là cột phá vỡ mọi phép nối.

Trung bình che giấu đúng những chỗ nghiêm trọng nhất. Đây là cùng một lỗi tư duy với việc dùng độ chính xác tổng thể cho dữ liệu mất cân bằng ở track AI.

Cách đúng: bảng đo có một dòng cho mỗi cột, và cột nào dưới ngưỡng của nó thì được tô đánh dấu. Ngưỡng đặt theo từng cột, vì email thiếu là bình thường còn khoá ngoại thiếu là nghiêm trọng.

B2. Mã mẫu — bộ đo ba chiều, sinh ra báo cáo

src/quality/three_dimensions.py

```
import pandas as pd
from dataclasses import dataclass, asdict

@dataclass
class Do:
    chieu: str        # DAY_DU | CHINH_XAC | NHAT_QUAN
    doi_tuong: str    # tên bảng.cột hoặc tên quy tắc
    so_kiem: int
    so_dat: int
    nguong: float
    @property
    def ti_le(self):  return self.so_dat / self.so_kiem if self.so_kiem else 0.0
    @property
    def dat(self):    return self.ti_le >= self.nguong

# ---------- CHIỀU 1: ĐẦY ĐỦ - đo theo TỪNG CỘT ----------
NGUONG_DAY_DU = {          # đặt riêng cho từng cột, KHÔNG dùng một ngưỡng chung
    'record_id': 1.0, 'ho_ten': 0.999, 'so_dien_thoai': 0.90,
    'email': 0.0,          # thiếu email là bình thường -> không đặt ngưỡng
    'dia_chi': 0.80, 'ngay_tao': 0.999,
}
def do_day_du(df, ten_bang):
    kq = []
    for cot, nguong in NGUONG_DAY_DU.items():
        if cot not in df.columns:
            continue
        co_gia_tri = df[cot].notna() & (df[cot].astype(str).str.strip() != '')
        kq.append(Do('DAY_DU', f'{ten_bang}.{cot}',
                     len(df), int(co_gia_tri.sum()), nguong))
    return kq

# ---------- CHIỀU 2: CHÍNH XÁC - cần một QUY TẮC để đối chiếu ----------
def do_chinh_xac(df_items, df_orders, df_tickets):
    kq = []
    # QT: thanh_tien = so_luong x don_gia (sai số làm tròn 1 đồng)
    khop = (df_items['thanh_tien'] - df_items['so_luong'] * df_items['don_gia']) \
            .abs() <= 1
    kq.append(Do('CHINH_XAC', 'order_items.thanh_tien = so_luong x don_gia',
                 len(df_items), int(khop.sum()), 0.98))

    # QT-02: số điện thoại chuẩn hoá được thành 10 chữ số
    co_sdt = df_orders['so_dien_thoai'].notna()
    chuan  = df_orders.loc[co_sdt, 'sdt_chuan'].notna()
    kq.append(Do('CHINH_XAC', 'orders.so_dien_thoai chuẩn hoá được',
                 int(co_sdt.sum()), int(chuan.sum()), 0.90))

    # QT: closed_at >= received_at  (không thể đóng phiếu trước khi nhận)
    da_dong = df_tickets['closed_at'].notna()
    hop_le  = df_tickets.loc[da_dong, 'closed_at'] >= \
              df_tickets.loc[da_dong, 'received_at']
    kq.append(Do('CHINH_XAC', 'tickets.closed_at >= received_at',
                 int(da_dong.sum()), int(hop_le.sum()), 1.0))
    return kq

# ---------- CHIỀU 3: NHẤT QUÁN - giữa các bảng và trong một cột ----------
GIA_TRI_CHO_PHEP = {'priority': {'CAO', 'TRUNG_BINH', 'THAP'}}

def do_nhat_quan(df_items, df_orders, df_tickets):
    kq = []
    # Toàn vẹn tham chiếu: mỗi dòng hàng trỏ về một đơn có thật
    hop_le = df_items['order_id'].isin(set(df_orders['order_id']))
    kq.append(Do('NHAT_QUAN', 'order_items.order_id -> orders.order_id',
                 len(df_items), int(hop_le.sum()), 1.0))

    # Khoá nghiệp vụ duy nhất: cặp (ma_cua_hang, ma_don)
    khong_trung = ~df_orders.duplicated(subset=['ma_cua_hang', 'ma_don'], keep=False)
    kq.append(Do('NHAT_QUAN', 'orders (ma_cua_hang, ma_don) duy nhất',
                 len(df_orders), int(khong_trung.sum()), 1.0))

    # Tập giá trị cho phép của cột phân loại
    for cot, cho_phep in GIA_TRI_CHO_PHEP.items():
        if cot in df_tickets.columns:
            trong_tap = df_tickets[cot].isin(cho_phep)
            kq.append(Do('NHAT_QUAN', f'tickets.{cot} thuộc tập cho phép',
```

```
                        len(df_tickets), int(trong_tap.sum()), 1.0))
    return kq

# ---------- Sinh báo cáo ----------
def sinh_bao_cao(tat_ca_do, duong_dan='reports/chat-luong.md'):
    dong = ['| Chiều | Đối tượng | Số kiểm | Số đạt | Tỉ lệ | Ngưỡng | Kết luận |',
            '|---|---|---:|---:|---:|---:|---|']
    for d in tat_ca_do:
        dong.append(f'| {d.chieu} | {d.doi_tuong} | {d.so_kiem} | {d.so_dat} | '
                    f'{d.ti_le:.2%} | {d.nguong:.2%} | '
                    f'{"ĐẠT" if d.dat else "**KHÔNG ĐẠT**"} |')
    khong_dat = [d for d in tat_ca_do if not d.dat]
    dong.append('')
    dong.append(f'**Tổng: {len(tat_ca_do)} phép đo, {len(khong_dat)} không đạt.**')
    with open(duong_dan, 'w', encoding='utf-8') as f:
        f.write('\n'.join(dong))
    return khong_dat
```

B3. Từ một phép đo KHÔNG ĐẠT tới một mục Bug Log

Một phép đo không đạt chưa phải là bug. Em phải điều tra để biết nó là bug của pipeline, là vấn đề của dữ liệu nguồn, hay là ngưỡng em đặt sai. Ba khả năng, ba cách xử lý khác nhau.

Khả năng	Dấu hiệu nhận biết	Cách xử lý	Ghi vào đâu
Bug của pipeline	Dữ liệu nguồn đúng nhưng dữ liệu ra sai; hoặc số dòng ra ít hơn số dòng vào mà không có dòng nào trong bảng reject	Sửa mã pipeline, chạy lại, đo lại	Bug Log — đây là bug thật, đủ bảy dòng
Vấn đề của dữ liệu nguồn	Dữ liệu nguồn đã sai từ đầu; pipeline xử lý đúng bằng cách đưa vào bảng reject	KHÔNG sửa dữ liệu. Ghi nhận, báo cáo tỉ lệ, và nêu rõ ảnh hưởng tới báo cáo nào	Báo cáo chất lượng + mục "hạn chế đã biết" trong tài liệu
Ngưỡng đặt sai	Ngưỡng đặt bằng 100% cho một cột mà bản chất nghiệp vụ cho phép thiếu (ví dụ email)	Sửa ngưỡng và ghi lý do vì sao ngưỡng đó là hợp lý	docs/thay-doi-thiet-ke.md kèm lý do

Ba câu trả lời khi giảng viên hỏi về một phép đo không đạt

Câu trả lời yếu: "Dòng đó dữ liệu bị lỗi nên em bỏ."

Câu trả lời tốt: "Phép đo CHINH_XAC về thanh_tien đạt 98,5%, dưới ngưỡng 99% em đặt ban đầu. Em điều tra: 534 trên 35.697 dòng có thanh_tien lệch so với so_luong × don_gia, lệch từ 10 nghìn tới 2 triệu đồng. Đây là vấn đề của dữ liệu nguồn, không phải bug pipeline — em đã kiểm bằng cách đọc lại tệp gốc. Em đưa 534 dòng đó vào bảng reject kèm cả hai giá trị, KHÔNG tự sửa số tiền vì dữ liệu tiền phải do người có thẩm quyền quyết định. Em hạ ngưỡng xuống 98% và ghi lý do; đồng thời ghi vào mục hạn chế đã biết rằng báo cáo doanh thu chi tiết theo dòng hàng có sai lệch tối đa 1,5% số dòng."

Điểm khác biệt: câu trả lời tốt có SỐ, có kết luận về loại vấn đề, có lý do KHÔNG sửa, và có nêu ảnh hưởng nghiệp vụ. Đó là ba nguyên tắc của học phần thể hiện trong một câu trả lời.

B4. Checklist tự kiểm trước khi tới workshop buổi 14 [DA]

- ☐ Đo ĐẦY ĐỦ theo từng cột, mỗi cột có ngưỡng riêng và có lý do cho ngưỡng đó.
- ☐ Đo CHÍNH XÁC với ít nhất ba quy tắc nghiệp vụ cụ thể.

- ☐ Đo NHẤT QUÁN với ít nhất ba phép kiểm: toàn vẹn tham chiếu, khoá duy nhất, tập giá trị cho phép.
- ☐ Báo cáo chất lượng sinh ra tự động sau mỗi lần chạy, có bảng đủ bảy cột.
- ☐ Mỗi phép đo không đạt đã được điều tra và phân loại thành một trong ba khả năng ở mục B3.
- ☐ Bug thật của pipeline đã được ghi vào Bug Log đủ bảy dòng, có link commit.
- ☐ Vấn đề của dữ liệu nguồn đã được ghi vào mục "hạn chế đã biết" kèm ảnh hưởng nghiệp vụ.
- ☐ Không tự sửa dữ liệu tiền hay dữ liệu định danh; mọi dòng không xử lý được đều nằm ở bảng reject.
- ☐ Tổng số dòng reject khớp với tổng số dòng bị loại nêu trong báo cáo chất lượng.
- ☐ Đã kiểm quyền truy cập: dashboard không cho xem cột dữ liệu cá nhân chưa cần thiết.

PHẦN C — Track AI: Kiểm định chéo, confusion matrix và đo thiên lệch

C1. Kiểm định chéo — khi nào cần và cần bao nhiêu

Với vài nghìn dòng dữ liệu, kết quả đo trên một lần chia dữ liệu dao động khá lớn: chia lại với hạt giống khác có thể ra chỉ số chênh vài điểm phần trăm. Kiểm định chéo cho em một con số ổn định hơn kèm độ dao động của nó.

src/evaluate.py — kiểm định chéo phân tầng 5 phần

```
import numpy as np
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression


def kiem_dinh_cheo(van_ban, nhan, hat_giong=42, so_phan=5):
    """Dùng Pipeline để vectorizer được fit RIÊNG trong từng phần.
       Nếu fit vectorizer ngoài vòng lặp thì lại là RÒ RỈ DỮ LIỆU."""
    pipe = Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1, 2), min_df=2)),
        ('clf',   LogisticRegression(max_iter=1000, class_weight='balanced',
                                     random_state=hat_giong)),
    ])
    cv = StratifiedKFold(n_splits=so_phan, shuffle=True, random_state=hat_giong)
    diem = cross_val_score(pipe, van_ban, nhan, cv=cv,
                           scoring='f1_macro', n_jobs=-1)
    return {
        'so_phan': so_phan,
        'diem_tung_phan': [round(float(d), 4) for d in diem],
        'trung_binh':     round(float(diem.mean()), 4),
        'do_lech_chuan':  round(float(diem.std()), 4),
        # Báo cáo dạng "trung bình +/- độ lệch" thay vì một con số trần
        'bao_cao':        f'{diem.mean():.3f} +/- {diem.std():.3f}',
    }
```

Ba điều phải đúng khi làm kiểm định chéo

1. Dùng Pipeline, không fit vectorizer trước vòng lặp. Nếu em fit vectorizer trên toàn bộ dữ liệu rồi mới chia thành 5 phần, mỗi phần kiểm đã bị nhiễm từ vựng của các phần khác. Kết quả cao hơn thực tế và vô nghĩa — đúng cùng một lỗi rò rỉ dữ liệu của buổi 7.

2. Dùng chia phân tầng (StratifiedKFold), không dùng KFold thường. Chia thường có thể tạo ra một phần không có mẫu nào của nhãn ít gặp, làm chỉ số của phần đó vô nghĩa.

3. Báo cáo cả độ lệch chuẩn. "F1 macro = 0,762 ± 0,031" nói được nhiều hơn "F1 macro = 0,762". Nếu độ lệch lớn hơn khoảng cách giữa hai mô hình em so sánh, thì em KHÔNG kết luận được mô hình nào tốt hơn — đó là một kết luận trung thực và đáng điểm.

C2. Đo thiên lệch giữa các nhóm — bắt buộc ở Bài tập 3

Một chỉ số tổng thể tốt vẫn có thể che giấu việc mô hình hoạt động kém hẳn với một nhóm dữ liệu nào đó. Với bộ dữ liệu Mekong Mobile, các nhóm đáng đo là: theo trung tâm bảo hành · theo nhóm sản phẩm · theo độ dài mô tả sự cố · theo việc còn hay hết bảo hành.

src/evaluate.py — đo chỉ số tách theo từng nhóm

```
import pandas as pd
from sklearn.metrics import f1_score, recall_score

def do_theo_nhom(df_test, y_that, y_du_doan, cot_nhom, so_mau_toi_thieu=30):
    """Đo F1 macro và recall cho từng nhóm. Bỏ qua nhóm quá ít mẫu
       vì số liệu trên nhóm dưới 30 mẫu không đáng tin."""
    kq = []
    d = df_test.assign(_that=y_that, _du_doan=y_du_doan)
    for ten_nhom, nhom in d.groupby(cot_nhom):
        if len(nhom) < so_mau_toi_thieu:
            kq.append({'nhom': str(ten_nhom), 'so_mau': len(nhom),
                       'ghi_chu': 'không đo - dưới ngưỡng mẫu tối thiểu'})
            continue
        kq.append({
            'nhom': str(ten_nhom),
            'so_mau': len(nhom),
            'f1_macro': round(float(f1_score(nhom['_that'], nhom['_du_doan'],
                                average='macro', zero_division=0)), 4),
            'recall_macro': round(float(recall_score(nhom['_that'], nhom['_du_doan'],
                                average='macro', zero_division=0)), 4),
        })
    bang = pd.DataFrame(kq)

    # CHÊNH LỆCH giữa nhóm tốt nhất và nhóm kém nhất - đây là con số phải báo cáo
    co_do = bang.dropna(subset=['f1_macro']) if 'f1_macro' in bang else bang
    if len(co_do) >= 2:
        chenh = co_do['f1_macro'].max() - co_do['f1_macro'].min()
        return bang, round(float(chenh), 4)
    return bang, None
```

Ngưỡng phán xét: chênh lệch bao nhiêu thì phải báo cáo

Chênh lệch F1 macro giữa nhóm tốt nhất và nhóm kém nhất dưới 0,05: ghi nhận là đã đo, không cần hành động thêm.

Chênh lệch từ 0,05 tới 0,15: phải nêu rõ trong báo cáo và trong Model card; nêu giả thuyết nguyên nhân (thường là nhóm đó ít mẫu huấn luyện hơn, hoặc ngôn ngữ mô tả khác).

Chênh lệch trên 0,15: đây là thiên lệch đáng kể. Phải nêu rõ trong Model card ở mục giới hạn sử dụng, và phải nêu một hướng giảm thiểu cụ thể (thu thêm mẫu cho nhóm đó, điều chỉnh trọng số lớp, hoặc hạ ngưỡng tin cậy riêng cho nhóm đó và chuyển sang người xem lại).

Nguyên tắc trung thực: đo được thiên lệch và báo cáo trung thực là ĐIỂM CỘNG. Không đo, hoặc đo rồi không báo cáo, là vấn đề liêm chính — và là chính xác tình huống của câu 4 đề thi cuối kỳ.

C3. Test cho một hệ thống học máy — bốn loại test

Một dự án học máy vẫn cần test theo nghĩa thông thường. Bốn loại dưới đây là bốn loại bắt buộc có trong Test Report của Bài tập 3 cho track AI.

Loại test	Kiểm cái gì	Ví dụ cụ thể
1. Test hàm biến đổi dữ liệu	Hàm làm sạch, hàm chuẩn hoá, hàm tạo đặc trưng — đây là hàm thuần nên test rẻ và ổn định	lam_sach("SN10293A màn hình vỡ") phải trả về chuỗi có thẻ <SERIAL> và giữ nguyên dấu tiếng Việt
2. Test chống rò rỉ dữ liệu	Vectorizer không được fit trên tập test	Sau khi huấn luyện, kiểm rằng từ vựng của vectorizer không chứa một từ chỉ xuất hiện trong tập test
3. Test tính tái lập	Hai lần chạy với cùng hạt giống cho cùng kết quả	Chạy huấn luyện hai lần và so sánh hai tệp chỉ số; phải giống hệt nhau

Loại test	Kiểm cái gì	Ví dụ cụ thể
4. Test dịch vụ dự đoán	Endpoint dự đoán xử lý đúng cả đầu vào hợp lệ và không hợp lệ	Mô tả hợp lệ → 200 kèm top 3 và độ tin cậy · mô tả rỗng hoặc quá ngắn → 422 · mô tả rất dài → không sập

tests/test_ml.py — bốn test theo bốn loại trên

```
import json, subprocess, pytest
from fastapi.testclient import TestClient
from src.features.text import lam_sach
from src.serve.app import app

# LOẠI 1 - test hàm biến đổi
def test_lam_sach_giu_dau_va_thay_serial():
    ra = lam_sach('SN10293A màn hình vỡ, gọi 0912345678')
    assert '<serial>' in ra.lower()
    assert '<sdt>' in ra.lower()
    assert 'màn hình' in ra          # PHẢI giữ dấu tiếng Việt
    assert '0912345678' not in ra    # KHÔNG được còn số điện thoại (QT-15)

# LOẠI 2 - test chống rò rỉ dữ liệu
def test_vectorizer_khong_biet_tu_vung_cua_tap_test(du_lieu_da_chia):
    train, val, test, vec = du_lieu_da_chia
    tu_vung = set(vec.vocabulary_.keys())
    tu_train = {t for s in train['mo_ta_sach'] for t in s.split()}
    tu_test  = {t for s in test['mo_ta_sach'] for t in s.split()}
    tu_chi_co_o_test = tu_test - tu_train        # so sánh TỪ, không so chuỗi con
    assert not (tu_vung & tu_chi_co_o_test), \
        'Vectorizer đã nhìn thấy từ vựng của tập test -> RÒ RỈ DỮ LIỆU'

# LOẠI 3 - test tính tái lập
def test_hai_lan_huan_luyen_cho_ket_qua_giong_nhau():
    lenh = ['python', '-m', 'src.models.train', '--config', 'configs/train.yaml']
    subprocess.run(lenh, check=True)
    lan1 = json.load(open('reports/metrics.json'))
    subprocess.run(lenh, check=True)
    lan2 = json.load(open('reports/metrics.json'))
    assert lan1 == lan2, 'Hai lần chạy cho kết quả khác nhau -> chưa tái lập được'

# LOẠI 4 - test dịch vụ dự đoán
client = TestClient(app)

def test_du_doan_hop_le_tra_ve_top3():
    r = client.post('/du-doan', json={'mo_ta': 'Máy sạc không vào điện, cắm sạc không lên'})
    assert r.status_code == 200
    body = r.json()
    assert len(body['top_3']) == 3
    assert 0.0 <= body['do_tin_cay'] <= 1.0
    assert 'phien_ban_mo_hinh' in body

@pytest.mark.parametrize('mo_ta,ky_vong', [('', 422), ('abc', 422), ('x' * 5000, 422)])
def test_du_doan_du_lieu_sai_tra_ve_422(mo_ta, ky_vong):
    assert client.post('/du-doan', json={'mo_ta': mo_ta}).status_code == ky_vong
```

C4. Từ confusion matrix tới Bug Log

Nhắc lại từ buổi 9: đọc confusion matrix cho ra ba kết luận. Ở Bài tập 3, mỗi kết luận đó phải dẫn tới MỘT trong hai thứ: một mục Bug Log (nếu là lỗi kỹ thuật sửa được), hoặc một mục trong "giới hạn đã biết" của Model card (nếu là hạn chế của dữ liệu hoặc phương pháp).

Phát hiện từ confusion matrix	Là bug hay là hạn chế?	Đi vào đâu
Một nhãn có F1 bằng 0 và không mẫu nào được dự đoán vào nhãn đó, dù nhãn đó có mẫu trong tập test	Có thể là BUG: kiểm xem nhãn đó có bị mất khi mã hoá nhãn hay bị lọc mất khi làm sạch không	Bug Log nếu là bug mã hoá nhãn; Model card nếu nhãn đó thực sự chỉ có 12 mẫu huấn luyện

Phát hiện từ confusion matrix	Là bug hay là hạn chế?	Đi vào đâu
Hai nhãn bị nhầm lẫn qua lại nhiều (như PIN và SẠC)	HẠN CHẾ: hai nhãn thực sự dùng ngôn ngữ giống nhau	Model card, mục giới hạn; kèm đề xuất nghiệp vụ (gộp nhãn hoặc thêm câu hỏi phụ khi tiếp nhận)
Nhãn có recall thấp và đó là nhãn tốn kém nhất khi bỏ sót	HẠN CHẾ có rủi ro nghiệp vụ	Model card, mục rủi ro; kèm biện pháp giảm thiểu như hạ ngưỡng tin cậy riêng cho nhãn đó
Chỉ số trên tập test thấp hơn hẳn chỉ số kiểm định chéo trên tập train	Có thể là BUG: kiểm xem tập test có bị xử lý khác tập train không (thiếu bước làm sạch chẳng hạn)	Bug Log — đây là loại bug rất hay gặp và rất dễ bỏ sót

C5. Checklist tự kiểm trước khi tới workshop buổi 14 [AI]

- ☐ Có kiểm định chéo phân tầng 5 phần, dùng Pipeline (vectorizer fit riêng trong từng phần).
- ☐ Báo cáo chỉ số dạng "trung bình ± độ lệch chuẩn", không chỉ một con số.
- ☐ Đã đo chỉ số tách theo ít nhất hai cách nhóm khác nhau, và ghi lại chênh lệch giữa nhóm tốt nhất và kém nhất.
- ☐ Chênh lệch trên 0,15 đã được nêu trong Model card kèm một hướng giảm thiểu cụ thể.
- ☐ Có đủ bốn loại test ở mục C3, chạy PASS bằng một lệnh.
- ☐ Test chống rò rỉ dữ liệu thực sự chạy và thực sự PASS.
- ☐ Test tính tái lập thực sự chạy huấn luyện hai lần và so sánh.
- ☐ Đã đọc confusion matrix và viết ba kết luận.
- ☐ Mỗi kết luận đã dẫn tới một mục Bug Log hoặc một mục trong giới hạn đã biết.
- ☐ Dịch vụ dự đoán từ chối đúng cách với đầu vào rỗng, quá ngắn và quá dài.

Mẫu Test Report và Bug Log — điền sẵn một dòng cho mỗi track

Dùng đúng cấu trúc tám cột dưới đây cho Bài tập 3. Mỗi track có một dòng ví dụ đã điền đầy đủ để em đối chiếu mức độ chi tiết cần đạt.

Mẫu Test Report

Mã	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	P/F
TC-06 [SE]	Có 2 tài khoản: A (id 1) và B (id 2). B có phiếu id 7 ở trạng thái DANG_SUA. Đã dựng bằng seeds/002_two_users.sql.	1. Đăng nhập bằng A, lấy token. 2. Gọi PATCH /api/tickets/7 với token của A, thân yêu cầu {"priority":"THAP"}.	token của A; ticket id 7 (thuộc B)	Mã 403, thân phản hồi {"error":"KHONG_CO_QUYEN"}. Phiếu 7 KHÔNG bị thay đổi.	Cao	Mã 200. Phiếu 7 đã bị đổi priority thành THAP.	FAIL
TC-11 [DA]	Kho dữ liệu đã nạp lô 20260701. Bảng fact_ticket có 7.800 dòng.	1. Chạy lại pipeline với đúng khoảng ngày 2026-01-01 → 2026-06-30. 2. Đếm số dòng fact_ticket.	tu_ngay=2026-01-01, den_ngay=2026-06-30	SELECT COUNT(*) FROM fact_ticket trả về đúng 7.800.	Cao	Trả về 7.800.	PASS
TC-18 [AI]	Đã huấn luyện mô hình v1 với hạt giống 42. Dịch vụ dự đoán đang chạy ở cổng 8080.	1. POST /du-doan với mô tả rỗng. 2. Ghi mã trạng thái và thân phản hồi.	{"mo_ta": ""}	Mã 422; thân phản hồi nêu rõ trường mo_ta không đạt độ dài tối thiểu 5 ký tự.	Trung bình	Mã 422, thông báo "String should have at least 5 characters", trường mo_ta.	PASS

Mẫu Bug Log — một mục hoàn chỉnh bảy dòng, tương ứng TC-06 FAIL ở trên

BUG-03 — Sửa được phiếu của người dùng khác

1. Mã và mức độ: BUG-03 · CRITICAL — lỗ hổng kiểm soát truy cập (A01 OWASP), dữ liệu bị thay đổi thật.

2. Phát hiện bởi: TC-06 (kiểm quyền truy cập theo chủ sở hữu).

3. Triệu chứng: Người dùng đăng nhập sửa được phiếu bảo hành của người dùng khác. API trả về 200 và dữ liệu bị thay đổi thật.

4. Cách tái lập: Đăng nhập tài khoản A, lấy token; gọi PATCH /api/tickets/7 (phiếu thuộc tài khoản B) với thân {"priority":"THAP"}; kiểm bảng tickets thấy dòng id 7 đã đổi.

5. Nguyên nhân: Hàm capNhatPhieu trong src/services/ticket.service.js chỉ kiểm phiếu có tồn tại, không kiểm phiếu có thuộc về người gọi hay không. Middleware xác thực đã gắn req.user nhưng service không nhận và không dùng thông tin đó.

6. Cách sửa: Truyền userId vào capNhatPhieu; thêm phép kiểm chủ sở hữu và ném LoiNghiepVu("KHONG_CO_QUYEN", …, 403); middleware lỗi đã ánh xạ sẵn mã này; cập nhật hai lệnh gọi hàm ở tầng routes.

7. Link commit và xác nhận: https://github.com/<tài-khoản>/<repo>/commit/9f2c1ab · chạy lại TC-06 → 403, phiếu 7 không đổi → PASS. Bổ sung TC-06b: chủ sở hữu thật vẫn sửa được phiếu của mình → PASS.

Chú ý dòng 6 và dòng 7: sửa bug không chỉ là sửa một dòng mã, mà là sửa mọi chỗ liên quan và thêm test bảo đảm bug không quay lại. TC-06b kiểm rằng bản sửa không làm hỏng luồng đúng — đây là chi tiết phân biệt bài điểm cao.

Ba việc làm ngay trong tuần này, trước workshop buổi 14

1. Viết Test Plan bốn mục trước khi viết test case nào. Phạm vi và phần chủ ý không kiểm · loại test và số lượng mỗi tầng · tiêu chí thành công đo được · môi trường và dữ liệu thử. Mất hai mươi phút và định hình toàn bộ phần còn lại.

2. Chạy đúng MỘT test case của track mình từ đầu tới cuối, ghi đủ tám cột và CHỤP ẢNH kết quả chạy — ảnh chụp là minh chứng bắt buộc ở mục 7 của PDF Bài tập 3. Làm được một cái thì chín cái còn lại chỉ là lặp lại.

3. Với track SE: kiểm ngay mục A01 Broken Access Control. Đây là lỗ hổng phổ biến nhất trong prototype sinh viên và cũng là mục dễ kiểm nhất — mất năm phút và thường phát hiện ra một bug Critical thật.

VAN LANG UNIVERSITY · FACULTY OF INFORMATION TECHNOLOGY
SPECIALIZED GRADUATION TOPIC 1 · SEMESTER 1, ACADEMIC YEAR 2026 – 2027

Self-study guide, Session 13 — Testing by specialization

Testing tools and API testing [SE] · Three dimensions of data quality: completeness, accuracy, consistency [DA] · Cross-validation, the confusion matrix and measuring bias [AI]. Each part has runnable sample code and a Test Report template with one row filled in.

The output of this guide is components 1, 2 and 3 of Assignment 3 — Test Plan, Test Report and Bug Log. Work on them in the Session 14 workshop; submit at the end of Session 16.

Prepared by: **Dr. Nguyễn Trí Hải** · Department of Software Engineering

For internal circulation · Self-study series for the specialization part (30% of Session 13 theory time). Bilingual edition: this is the English version. In the sample code, identifiers, user-facing strings and log messages stay in Vietnamese because the case-study system serves Vietnamese users; code comments are translated.

How to use this guide

Taught by the lecturer in class (Session 13)	Read by students in this guide
Why "I already tried it" is not evidence; the four sections of a short Test Plan; the eight mandatory columns of a test case; three sources for choosing test cases; boundary values; the seven lines of a Bug Log entry; the four debugging steps; bug severity.	The concrete tools and techniques of your own track: which testing tools to run and how to write tests [SE]; the three dimensions of data quality and how to measure each [DA]; cross-validation, reading the confusion matrix and measuring bias between groups [AI].

Four things that hold for all three tracks — read before your own part

1. A TEST CASE MUST BE REPRODUCIBLE BY SOMEONE ELSE. It must state the precondition, the concrete test data and the concrete expected result. "Check that the lookup function works correctly" is not a test case — it is a wish.

2. RECORD RESULTS HONESTLY. In Session 14 the lecturer picks a random test case you marked PASS and asks you to run it again on the spot. Recording a FAIL honestly and then fixing it earns the highest marks; recording a fake PASS is an integrity issue.

3. "ALMOST RIGHT" IS A FAIL. If the test expects 409 and the system returns 400, it is a FAIL, even though both are errors. Being lenient here is where students lose the most marks.

4. WHEN YOU HIT A BUG, LOG IT IMMEDIATELY — DO NOT BATCH. The cause of a bug stays clear for only about ten minutes after you fix it. Leave it until the end of the session and you will only remember "fixed something in some function", and the Bug Log will be thin.

Reminder: the eight mandatory columns of a test case and the seven lines of a Bug Log entry

The eight columns of a test case (Assignment 3, section 2.2)	The seven lines of a Bug Log entry
1. Test case ID (TC-01, TC-02…) 2. Precondition — the state of the data and the system before running 3. Steps — numbered, one action per step 4. Test data — CONCRETE values, not "some valid number" 5. Expected result — observable, measurable 6. Priority — High / Medium / Low The last two columns are filled in when you ACTUALLY RUN it: 7. Actual result — in concrete words 8. PASS / FAIL	1. Bug ID (BUG-01…) and severity (Critical / Major / Minor) 2. Detected by which test case 3. Symptom — what the user sees 4. How to reproduce — the minimal steps that make the bug appear again 5. Cause — the REAL cause, not the symptom 6. Fix — what was changed, in which file 7. Link to the fixing commit + confirmation that the test case was re-run and PASSED

PART A — SE track: Testing tools and API testing

A1. Three test tiers and a sensible ratio for a prototype

Tier	What it checks	Sensible number for Assignment 3	Cost to write and maintain
Unit test	One function or class, isolated from every external dependency (using test doubles)	6 – 8 tests, focused on the service layer	Cheapest, fastest (milliseconds), most stable
Integration test	Several real components working together, usually including a real database	2 – 3 tests for the most important flows	Medium; needs test data to be set up and cleaned
API / end-to-end test	The whole system through its external interface (HTTP), like a real user	2 – 3 tests for the main business flow	Most expensive, slowest, most brittle when the system changes

Why this ratio and not the reverse

Unit tests are cheap, so you can write many; they catch most business-logic bugs and point straight at the failing function.

End-to-end tests are expensive, so write few; they catch integration bugs unit tests cannot see, but when one turns red you still have to hunt for the cause.

A common mistake: writing only UI tests. They are slow, break for unrelated reasons, and when they break they cannot tell you where the bug is.

For Assignment 3, 10–12 test cases in this ratio are enough. The number that matters more: at least one test for EVERY business rule you have implemented.

A2. Tools per stack — choose one and put the run command in the README

Stack	Suggested tools	Run command	Notes
Node.js / Express	Jest or Vitest, with Supertest for API testing	npm test	Supertest can call the Express app without opening a real port — faster and more stable than calling over HTTP.
Java / Spring Boot	JUnit 5 + Mockito, with MockMvc or @SpringBootTest	mvn test or ./gradlew test	MockMvc tests the web layer without starting a full server.
Python / FastAPI, Flask, Django	pytest, with FastAPI's TestClient or Django's test client	pytest -q	pytest fixtures are the best tool for setting up and cleaning test data.
Manual API testing (any stack)	An exported Postman collection, or a REST Client .http file	Run in Postman, or newman run <file>	It must be exported and committed to the repo — a collection that only lives on your machine is not evidence.

A3. Sample code — three tests for one business rule

tests/warranty.test.js — Jest, testing the warranty rule QT-05

```
const TicketService = require('../src/services/ticket.service');

describe('QT-05 xác định còn hạn bảo hành', () => {
  const svc = new TicketService(null, null, null);   // pure function, no repository needed

  // TC-01 HAPPY PATH
  test('TC-01 thiết bị bán cách đây 3 tháng, bảo hành 12 tháng -> còn hạn', () => {
    expect(svc.isUnderWarranty('2026-06-01', 12, new Date('2026-09-01'))).toBe(true);
  });

  // TC-02 BOUNDARY VALUES - two tests on either side of the boundary
  test.each([
    ['2027-03-01', true,  'đúng ngày hết hạn -> VẪN còn bảo hành'],
    ['2027-03-02', false, 'sau ngày hết hạn một ngày -> HẾT bảo hành'],
  ])('TC-02 biên: %s -> %s (%s)', (ngayKiem, kyVong) => {
    expect(svc.isUnderWarranty('2026-03-01', 12, new Date(ngayKiem))).toBe(kyVong);
  });

  // TC-03 INVALID DATA
  test('TC-03 ngày bán không đọc được -> ném lỗi, KHÔNG trả về true', () => {
    expect(() => svc.isUnderWarranty('khong-phai-ngay', 12, new Date()))
      .toThrow();
  });
});
```

tests/tickets.api.test.js — Supertest, API testing over HTTP

```
const request = require('supertest');
const { buildApp } = require('../src/app');
const { resetDb, seedDemo } = require('./helpers/db');

describe('POST /api/tickets', () => {
  let app;
  beforeEach(async () => {
    await resetDb();          // the PRECONDITION is built in code,
    await seedDemo();         // not dependent on data already on the machine
    app = buildApp();
  });

  test('TC-04 tạo phiếu thành công trả về 201 và có ticket_code', async () => {
    const res = await request(app).post('/api/tickets').send({
      customerId: 1, serialNo: 'SN10293A', productId: 5, centerId: 2,
      categoryId: 1, priority: 'CAO',
    });
    expect(res.status).toBe(201);
    expect(res.body.data.ticket_code).toMatch(/^BH-\d{6}\/\d{4}$/);   // e.g. BH-000231/2026
  });

  test('TC-05 thiếu trường bắt buộc -> 400 và thông báo NÊU RÕ trường nào', async () => {
    const res = await request(app).post('/api/tickets').send({ customerId: 1 });
    expect(res.status).toBe(400);
    expect(res.body.message).toMatch(/serialNo/);      // the message must name the field
  });

  test('TC-06 serial đã có phiếu đang mở -> 409, KHÔNG phải 400 hay 500 (QT-03)', async () => {
    await request(app).post('/api/tickets').send({
      customerId: 1, serialNo: 'SN10293A', productId: 5, centerId: 2,
      categoryId: 1, priority: 'CAO' });
    const res = await request(app).post('/api/tickets').send({
      customerId: 1, serialNo: 'SN10293A', productId: 5, centerId: 2,
      categoryId: 1, priority: 'CAO' });
    expect(res.status).toBe(409);
    expect(res.body.error).toBe('PHIEU_DANG_MO');
  });
});
```

Three things the code above shows — you will be asked about them in Session 14

1. The precondition is built in code, not assumed. resetDb() and seedDemo() make the test run on any machine. A test that depends on data already on your machine is not evidence.

2. Check both the STATUS CODE and the CONTENT. TC-05 checks not only 400 but also that the message names the missing field. A useless error message still returns 400.

3. Tell 400, 409 and 500 apart. 400 is malformed input · 409 is a conflict with business state · 500 is an unexpected error. A system that returns 500 for a known business error has a badly designed error handling.

A4. Security testing at awareness level — five priority items from the OWASP Top 10:2025

In Session 13 you only need to be ABLE TO TEST the five items below by hand and record the results. A full security review belongs to Specialized Graduation Topic 2. The list follows the OWASP Top 10:2025.

OWASP item	What it means	How to test it yourself by hand	Expected result
A01:2025 Broken Access Control	A user can reach something that is not theirs	Log in as account A, take the id of a record owned by account B, call the API to update or delete that record	403 or 404, NEVER 200. This is the most important item and the one most often missed.
A02:2025 Security Misconfiguration	Configuration that leaks information or is too permissive	Call an endpoint that causes an error and read the response; check whether CORS is open to every origin	The error response contains NO stack trace, table names or connection string. CORS is only open to the origins needed.
A04:2025 Cryptographic Failures	Sensitive data is not properly protected	Query the users table directly and look at the password column	It must be a hash (bcrypt, argon2), never a readable password.
A05:2025 Injection	User data is interpreted as code	Type into a search field the string ' OR 1=1 -- and a script tag string	Not every record is returned; the tag string is displayed as text, not executed.
A07:2025 Authentication Failures	The authentication mechanism has holes	Call an endpoint that needs login without a token; with an expired token; with a token altered by one character	All three cases must return 401. A token altered by one character that still works is a serious bug.

How to record security checks in the Test Report

Each item is a full test case with all eight columns, like any other. Do not write "security checked, passed".

A correct example: TC-09 · Precondition: two accounts A and B, B owns ticket id=7 · Steps: log in as A, call PATCH /api/tickets/7 · Test data: A's token, id 7 · Expected: 403 · Actual: 403, message "Không có quyền" · PASS.

If you find a real vulnerability: log a Critical Bug Log entry immediately, fix it and record the commit link. Finding a hole in your own system is a plus, not a minus.

If an item does not apply: write "not applicable" with a one-line reason, e.g. "the system has no login yet, so A07 does not apply to this version".

A5. Self-check before the Session 14 workshop [SE]

- ☐ A testing tool is configured and one single command runs everything; it is in the README.
- ☐ At least one test for EVERY business rule implemented.
- ☐ At least two boundary-value tests (both sides of one boundary).
- ☐ At least two invalid-data tests, checking both the status code and the message content.
- ☐ Tests build their preconditions in code and do not depend on data already on the machine.
- ☐ A Postman collection or .http file has been exported and committed.
- ☐ You have checked the five OWASP items in A4 and recorded the result of each.
- ☐ Error responses do not reveal stack traces, table names or connection strings.
- ☐ Passwords in the database are hashes, not readable.
- ☐ At least one real Bug Log entry exists, with a commit link that opens.

PART B — DA track: The three dimensions of data quality

B1. Three quality dimensions — definition and measurement

For the DA track, "testing" means measuring the quality of the pipeline's output data, not only checking the source code. The three dimensions below are mandatory measurements in Assignment 3.

Dimension	The question it answers	How it is measured	Suggested threshold for the Mekong Mobile dataset
COMPLETENESS	How much data is missing, and where?	Per column: rows with a value / total rows. Measure PER COLUMN, not as an average over the table.	so_dien_thoai ≥ 90% (the source is missing ~6.1%) · email has no threshold (many missing is normal) · every key and date column ≥ 99.9%
ACCURACY	Does the data match reality or a known rule?	Rows satisfying the rule / rows checked. Needs a rule or a reference source.	thanh_tien = so_luong × don_gia ≥ 98% (the source deviates ~1.5%) · phone numbers normalisable ≥ 90% · closed_at ≥ received_at = 100%
CONSISTENCY	Is the same entity represented the same way everywhere?	Consistent pairs/groups / groups checked. Usually between two tables or within a categorical column.	Every order_id in order_items exists in orders = 100% · pair (ma_cua_hang, ma_don) unique = 100% · priority only takes the three allowed values = 100%

Why COMPLETENESS must be measured per column, not averaged over the table

A table with 20 columns: 19 are 100% full and the foreign-key column is 30% missing → the table average is 98.5%, which sounds great. But the 30%-missing column breaks every join.

Averages hide exactly the most serious problems. It is the same thinking error as using overall accuracy on imbalanced data in the AI track.

The right way: the measurement table has one row per column, and every column below its threshold is flagged. Thresholds are set per column, because missing email is normal while a missing foreign key is serious.

B2. Sample code — a three-dimension measurer that produces a report

src/quality/three_dimensions.py

```
import pandas as pd
from dataclasses import dataclass, asdict

@dataclass
class Do:
    chieu: str        # DAY_DU | CHINH_XAC | NHAT_QUAN
    doi_tuong: str    # table.column or rule name
    so_kiem: int
    so_dat: int
    nguong: float
    @property
    def ti_le(self):  return self.so_dat / self.so_kiem if self.so_kiem else 0.0
    @property
    def dat(self):    return self.ti_le >= self.nguong

# ---------- DIMENSION 1: COMPLETENESS - measured PER COLUMN ----------
NGUONG_DAY_DU = {          # set per column, NOT one shared threshold
    'record_id': 1.0, 'ho_ten': 0.999, 'so_dien_thoai': 0.90,
    'email': 0.0,          # missing email is normal -> no threshold
    'dia_chi': 0.80, 'ngay_tao': 0.999,
}
def do_day_du(df, ten_bang):
    kq = []
    for cot, nguong in NGUONG_DAY_DU.items():
        if cot not in df.columns:
            continue
        co_gia_tri = df[cot].notna() & (df[cot].astype(str).str.strip() != '')
        kq.append(Do('DAY_DU', f'{ten_bang}.{cot}',
                     len(df), int(co_gia_tri.sum()), nguong))
    return kq

# ---------- DIMENSION 2: ACCURACY - needs a RULE to compare against ----------
def do_chinh_xac(df_items, df_orders, df_tickets):
    kq = []
    # Rule: thanh_tien = so_luong x don_gia (rounding tolerance 1 dong)
    khop = (df_items['thanh_tien'] - df_items['so_luong'] * df_items['don_gia']) \
             .abs() <= 1
    kq.append(Do('CHINH_XAC', 'order_items.thanh_tien = so_luong x don_gia',
                 len(df_items), int(khop.sum()), 0.98))

    # QT-02: phone numbers can be normalised to 10 digits
    co_sdt = df_orders['so_dien_thoai'].notna()
    chuan  = df_orders.loc[co_sdt, 'sdt_chuan'].notna()
    kq.append(Do('CHINH_XAC', 'orders.so_dien_thoai chuẩn hoá được',
                 int(co_sdt.sum()), int(chuan.sum()), 0.90))

    # Rule: closed_at >= received_at  (a ticket cannot close before it is received)
    da_dong = df_tickets['closed_at'].notna()
    hop_le  = df_tickets.loc[da_dong, 'closed_at'] >= \
              df_tickets.loc[da_dong, 'received_at']
    kq.append(Do('CHINH_XAC', 'tickets.closed_at >= received_at',
                 int(da_dong.sum()), int(hop_le.sum()), 1.0))
    return kq

# ---------- DIMENSION 3: CONSISTENCY - across tables and within a column ----------
GIA_TRI_CHO_PHEP = {'priority': {'CAO', 'TRUNG_BINH', 'THAP'}}

def do_nhat_quan(df_items, df_orders, df_tickets):
    kq = []
    # Referential integrity: every order line points to a real order
    hop_le = df_items['order_id'].isin(set(df_orders['order_id']))
    kq.append(Do('NHAT_QUAN', 'order_items.order_id -> orders.order_id',
                 len(df_items), int(hop_le.sum()), 1.0))

    # Unique business key: the pair (ma_cua_hang, ma_don)
    khong_trung = ~df_orders.duplicated(subset=['ma_cua_hang', 'ma_don'], keep=False)
    kq.append(Do('NHAT_QUAN', 'orders (ma_cua_hang, ma_don) duy nhất',
                 len(df_orders), int(khong_trung.sum()), 1.0))

    # Allowed value set of a categorical column
    for cot, cho_phep in GIA_TRI_CHO_PHEP.items():
        if cot in df_tickets.columns:
            trong_tap = df_tickets[cot].isin(cho_phep)
            kq.append(Do('NHAT_QUAN', f'tickets.{cot} thuộc tập cho phép',
```

```
                        len(df_tickets), int(trong_tap.sum()), 1.0))
    return kq

# ---------- Produce the report ----------
def sinh_bao_cao(tat_ca_do, duong_dan='reports/chat-luong.md'):
    dong = ['| Chiều | Đối tượng | Số kiểm | Số đạt | Tỉ lệ | Ngưỡng | Kết luận |',
            '|---|---|---:|---:|---:|---:|---|']
    for d in tat_ca_do:
        dong.append(f'| {d.chieu} | {d.doi_tuong} | {d.so_kiem} | {d.so_dat} | '
                    f'{d.ti_le:.2%} | {d.nguong:.2%} | '
                    f'{"ĐẠT" if d.dat else "**KHÔNG ĐẠT**"} |')
    khong_dat = [d for d in tat_ca_do if not d.dat]
    dong.append('')
    dong.append(f'**Tổng: {len(tat_ca_do)} phép đo, {len(khong_dat)} không đạt.**')
    with open(duong_dan, 'w', encoding='utf-8') as f:
        f.write('\n'.join(dong))
    return khong_dat
```

B3. From a FAILED measurement to a Bug Log entry

A failed measurement is not yet a bug. You must investigate whether it is a pipeline bug, a problem in the source data, or a threshold you set wrongly. Three possibilities, three different ways of handling them.

Possibility	How to recognise it	How to handle it	Where to record it
A pipeline bug	The source data is correct but the output is wrong; or fewer rows come out than went in, with nothing in the reject table	Fix the pipeline code, re-run, re-measure	Bug Log — this is a real bug, all seven lines
A source-data problem	The source data was wrong from the start; the pipeline handles it correctly by sending it to the reject table	Do NOT fix the data. Record it, report the rate, and state which reports are affected	Quality report + the "known limitations" section of the documentation
A wrong threshold	A threshold of 100% on a column whose business nature allows missing values (e.g. email)	Correct the threshold and record why the new threshold is reasonable	docs/thay-doi-thiet-ke.md with the reason

Three answers when the lecturer asks about a failed measurement

A weak answer: "That row had bad data so I dropped it."

A good answer: "The ACCURACY measurement for thanh_tien reached 98.5%, below the 99% I first set. I investigated: 534 of 35,697 rows have a thanh_tien that differs from so_luong × don_gia, by 10 thousand to 2 million dong. It is a source-data problem, not a pipeline bug — I checked by re-reading the original file. I put those 534 rows in the reject table with both values and did NOT correct the amounts myself, because money data must be decided by someone with authority. I lowered the threshold to 98% and recorded why, and noted under known limitations that line-level revenue reports are off for up to 1.5% of rows."

The difference: the good answer has NUMBERS, a conclusion about the type of problem, a reason for NOT fixing, and the business impact. That is the course's three principles in a single answer.

B4. Self-check before the Session 14 workshop [DA]

- ☐ COMPLETENESS is measured per column; each column has its own threshold with a reason.
- ☐ ACCURACY is measured with at least three concrete business rules.

- ☐ CONSISTENCY is measured with at least three checks: referential integrity, unique keys, allowed value sets.
- ☐ The quality report is generated automatically after every run, with a table of all seven columns.
- ☐ Every failed measurement has been investigated and classified as one of the three possibilities in B3.
- ☐ Real pipeline bugs are in the Bug Log with all seven lines and a commit link.
- ☐ Source-data problems are recorded under "known limitations" with their business impact.
- ☐ Money and identity data are never corrected by hand; every row that cannot be processed is in the reject table.
- ☐ The total number of reject rows matches the total removed rows stated in the quality report.
- ☐ Access has been checked: the dashboard does not show personal-data columns that are not needed.

PART C — AI track: Cross-validation, the confusion matrix and measuring bias

C1. Cross-validation — when you need it and how much

With a few thousand rows, a score measured on a single split varies a lot: splitting again with another seed can move the score by several percentage points. Cross-validation gives you a more stable number together with its spread.

src/evaluate.py — stratified 5-fold cross-validation

```
import numpy as np
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression


def kiem_dinh_cheo(van_ban, nhan, hat_giong=42, so_phan=5):
    """Use a Pipeline so the vectorizer is fitted SEPARATELY in each fold.
       Fitting the vectorizer outside the loop is DATA LEAKAGE again."""
    pipe = Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1, 2), min_df=2)),
        ('clf',   LogisticRegression(max_iter=1000, class_weight='balanced',
                                     random_state=hat_giong)),
    ])
    cv = StratifiedKFold(n_splits=so_phan, shuffle=True, random_state=hat_giong)
    diem = cross_val_score(pipe, van_ban, nhan, cv=cv,
                           scoring='f1_macro', n_jobs=-1)
    return {
        'so_phan': so_phan,
        'diem_tung_phan': [round(float(d), 4) for d in diem],
        'trung_binh':     round(float(diem.mean()), 4),
        'do_lech_chuan':  round(float(diem.std()), 4),
        # Report as "mean +/- std" rather than one bare number
        'bao_cao':        f'{diem.mean():.3f} +/- {diem.std():.3f}',
    }
```

Three things that must be right in cross-validation

1. Use a Pipeline; do not fit the vectorizer before the loop. If you fit the vectorizer on all the data and then split into 5 folds, every test fold is contaminated by the vocabulary of the others. The score is inflated and meaningless — the same data-leakage error as in Session 7.

2. Use stratified splitting (StratifiedKFold), not plain KFold. A plain split can create a fold with no example of a rare label, making that fold's score meaningless.

3. Report the standard deviation too. "Macro F1 = 0.762 ± 0.031" says more than "Macro F1 = 0.762". If the spread is larger than the gap between the two models you compare, you CANNOT conclude which model is better — an honest conclusion worth marks.

C2. Measuring bias between groups — mandatory in Assignment 3

A good overall score can hide a model that performs much worse for some group. For the Mekong Mobile dataset, the groups worth measuring are: by service centre · by product group · by length of the issue description · by whether the device is under warranty.

src/evaluate.py — scores split by group

```
import pandas as pd
from sklearn.metrics import f1_score, recall_score

def do_theo_nhom(df_test, y_that, y_du_doan, cot_nhom, so_mau_toi_thieu=30):
    """Measure macro F1 and recall per group. Skip groups with too few samples,
       because figures on a group under 30 samples are not reliable."""
    kq = []
    d = df_test.assign(_that=y_that, _du_doan=y_du_doan)
    for ten_nhom, nhom in d.groupby(cot_nhom):
        if len(nhom) < so_mau_toi_thieu:
            kq.append({'nhom': str(ten_nhom), 'so_mau': len(nhom),
                       'ghi_chu': 'không đo - dưới ngưỡng mẫu tối thiểu'})
            continue
        kq.append({
            'nhom': str(ten_nhom),
            'so_mau': len(nhom),
            'f1_macro': round(float(f1_score(nhom['_that'], nhom['_du_doan'],
                                average='macro', zero_division=0)), 4),
            'recall_macro': round(float(recall_score(nhom['_that'], nhom['_du_doan'],
                                average='macro', zero_division=0)), 4),
        })
    bang = pd.DataFrame(kq)

    # GAP between the best and the worst group - this is the number to report
    co_do = bang.dropna(subset=['f1_macro']) if 'f1_macro' in bang else bang
    if len(co_do) >= 2:
        chenh = co_do['f1_macro'].max() - co_do['f1_macro'].min()
        return bang, round(float(chenh), 4)
    return bang, None
```

Decision thresholds: how large a gap must be reported

Macro-F1 gap between the best and the worst group below 0.05: record that it was measured; no further action.

Gap from 0.05 to 0.15: state it clearly in the report and in the Model card, with a hypothesis about the cause (usually fewer training samples for that group, or different description language).

Gap above 0.15: this is significant bias. State it in the Model card under limitations of use, and give one concrete mitigation (collect more samples for that group, adjust class weights, or lower the confidence threshold for that group and route it to human review).

The honesty principle: measuring bias and reporting it honestly is a PLUS. Not measuring it, or measuring and not reporting, is an integrity issue — exactly the situation of question 4 of the final exam.

C3. Testing a machine-learning system — four kinds of test

A machine-learning project still needs tests in the ordinary sense. The four kinds below are mandatory in the AI track's Test Report for Assignment 3.

Kind of test	What it checks	Concrete example
1. Data-transformation function test	Cleaning, normalisation and feature functions — pure functions, so tests are cheap and stable	lam_sach("SN10293A màn hình vỡ") must return a string with a <SERIAL> token and keep the Vietnamese diacritics
2. Data-leakage test	The vectorizer must not be fitted on the test set	After training, check that the vectorizer's vocabulary contains no word that appears only in the test set

Kind of test	What it checks	Concrete example
3. Reproducibility test	Two runs with the same seed give the same result	Run training twice and compare the two metrics files; they must be identical
4. Prediction-service test	The prediction endpoint handles both valid and invalid input	Valid description → 200 with top 3 and confidence · empty or too short → 422 · very long → no crash

tests/test_ml.py — four tests for the four kinds above

```
import json, subprocess, pytest
from fastapi.testclient import TestClient
from src.features.text import lam_sach
from src.serve.app import app

# KIND 1 - transformation-function test
def test_lam_sach_giu_dau_va_thay_serial():
    ra = lam_sach('SN10293A màn hình vỡ, gọi 0912345678')
    assert '<serial>' in ra.lower()
    assert '<sdt>' in ra.lower()
    assert 'màn hình' in ra          # MUST keep Vietnamese diacritics
    assert '0912345678' not in ra    # the phone number must NOT remain (QT-15)

# KIND 2 - data-leakage test
def test_vectorizer_khong_biet_tu_vung_cua_tap_test(du_lieu_da_chia):
    train, val, test, vec = du_lieu_da_chia
    tu_vung = set(vec.vocabulary_.keys())
    tu_train = {t for s in train['mo_ta_sach'] for t in s.split()}
    tu_test  = {t for s in test['mo_ta_sach'] for t in s.split()}
    tu_chi_co_o_test = tu_test - tu_train        # compare WORDS, not substrings
    assert not (tu_vung & tu_chi_co_o_test), \
        'Vectorizer đã nhìn thấy từ vựng của tập test -> RÒ RỈ DỮ LIỆU'

# KIND 3 - reproducibility test
def test_hai_lan_huan_luyen_cho_ket_qua_giong_nhau():
    lenh = ['python', '-m', 'src.models.train', '--config', 'configs/train.yaml']
    subprocess.run(lenh, check=True)
    lan1 = json.load(open('reports/metrics.json'))
    subprocess.run(lenh, check=True)
    lan2 = json.load(open('reports/metrics.json'))
    assert lan1 == lan2, 'Hai lần chạy cho kết quả khác nhau -> chưa tái lập được'

# KIND 4 - prediction-service test
client = TestClient(app)

def test_du_doan_hop_le_tra_ve_top3():
    r = client.post('/du-doan', json={'mo_ta': 'Máy sạc không vào điện, cắm sạc không lên'})
    assert r.status_code == 200
    body = r.json()
    assert len(body['top_3']) == 3
    assert 0.0 <= body['do_tin_cay'] <= 1.0
    assert 'phien_ban_mo_hinh' in body

@pytest.mark.parametrize('mo_ta,ky_vong', [('', 422), ('abc', 422), ('x' * 5000, 422)])
def test_du_doan_du_lieu_sai_tra_ve_422(mo_ta, ky_vong):
    assert client.post('/du-doan', json={'mo_ta': mo_ta}).status_code == ky_vong
```

C4. From the confusion matrix to the Bug Log

Recall from Session 9: reading a confusion matrix yields three conclusions. In Assignment 3 each conclusion must lead to ONE of two things: a Bug Log entry (if it is a technical bug that can be fixed), or an entry under "known limitations" in the Model card (if it is a limitation of the data or the method).

Finding from the confusion matrix	Bug or limitation?	Where it goes
A label has F1 = 0 and nothing is predicted into it, although it has samples in the test set	Possibly a BUG: check whether the label was lost during label encoding or filtered out during cleaning	Bug Log if it is a label-encoding bug; Model card if the label really has only 12 training samples
Two labels are often confused with each other (such as PIN and SẠC)	LIMITATION: the two labels really use similar language	Model card, limitations; with a business proposal (merge the labels or add a follow-up question at intake)
A label has low recall and it is the costliest label to miss	A LIMITATION with business risk	Model card, risks; with a mitigation such as a lower confidence threshold for that label
The test-set score is much lower than the cross-validation score on the training set	Possibly a BUG: check whether the test set was processed differently from the training set (e.g. a missing cleaning step)	Bug Log — a very common bug that is easy to miss

C5. Self-check before the Session 14 workshop [AI]

- ☐ Stratified 5-fold cross-validation is in place, using a Pipeline (vectorizer fitted separately in each fold).
- ☐ Scores are reported as "mean ± standard deviation", not one number.
- ☐ Scores are split by at least two different groupings, and the gap between the best and worst group is recorded.
- ☐ A gap above 0.15 is stated in the Model card with one concrete mitigation.
- ☐ All four kinds of test in C3 exist and PASS with one command.
- ☐ The data-leakage test really runs and really PASSES.
- ☐ The reproducibility test really trains twice and compares.
- ☐ The confusion matrix has been read and three conclusions written.
- ☐ Each conclusion leads to a Bug Log entry or a known-limitations entry.
- ☐ The prediction service rejects empty, too-short and too-long input correctly.

Test Report and Bug Log templates — one filled-in row per track

Use exactly the eight-column structure below for Assignment 3. Each track has one fully completed example row so you can compare the level of detail required.

Test Report template

ID	Precondition	Steps	Test data	Expected result	Priority	Actual result	P/F
TC-06 [SE]	Two accounts: A (id 1) and B (id 2). B owns ticket id 7 in state DANG_SUA. Built with seeds/002_two_users.sql.	1. Log in as A, get the token. 2. Call PATCH /api/tickets/7 with A's token, body {"priority":"THAP"}.	A's token; ticket id 7 (owned by B)	Status 403, body {"error":"KHONG_CO_QUYEN"}. Ticket 7 is NOT changed.	High	Status 200. Ticket 7's priority was changed to THAP.	FAIL
TC-11 [DA]	The warehouse has loaded batch 20260701. Table fact_ticket has 7,800 rows.	1. Re-run the pipeline with exactly the range 2026-01-01 → 2026-06-30. 2. Count the rows of fact_ticket.	tu_ngay=2026-01-01, den_ngay=2026-06-30	SELECT COUNT(*) FROM fact_ticket returns exactly 7,800.	High	Returns 7,800.	PASS
TC-18 [AI]	Model v1 trained with seed 42. The prediction service runs on port 8080.	1. POST /du-doan with an empty description. 2. Record the status code and response body.	{"mo_ta": ""}	Status 422; the body says field mo_ta is below the minimum length of 5 characters.	Medium	Status 422, message "String should have at least 5 characters", field mo_ta.	PASS

Bug Log template — one complete seven-line entry, matching the failed TC-06 above

BUG-03 — A user can edit another user's ticket

1. ID and severity: BUG-03 · CRITICAL — an access-control hole (OWASP A01); data is really changed.

2. Detected by: TC-06 (owner-based access check).

3. Symptom: A logged-in user can edit another user's warranty ticket. The API returns 200 and the data really changes.

4. How to reproduce: Log in as A, get the token; call PATCH /api/tickets/7 (owned by B) with body {"priority":"THAP"}; the tickets table shows row id 7 changed.

5. Cause: capNhatPhieu in src/services/ticket.service.js only checks that the ticket exists, not that it belongs to the caller. The auth middleware sets req.user, but the service neither receives nor uses it.

6. Fix: Pass userId into capNhatPhieu; add an owner check that throws LoiNghiepVu("KHONG_CO_QUYEN", …, 403); the error middleware already maps this code; update the two call sites in the routes layer.

7. Commit link and confirmation: https://github.com/<account>/<repo>/commit/9f2c1ab · re-ran TC-06 → 403, ticket 7 unchanged → PASS. Added TC-06b: the real owner can still edit their ticket → PASS.

Note lines 6 and 7: fixing a bug is not changing one line of code, but fixing every related place and adding a test so the bug cannot come back. TC-06b checks that the fix does not break the happy path — the detail that distinguishes high-scoring work.

Three things to do this week, before the Session 14 workshop

1. Write the four-section Test Plan before writing any test case. Scope and what is deliberately not tested · test types and counts per tier · measurable success criteria · environment and test data. It takes twenty minutes and shapes everything else.

2. Run exactly ONE test case of your track from start to finish, fill in all eight columns and TAKE A SCREENSHOT of the run — screenshots are mandatory evidence in section 7 of the Assignment 3 PDF.

3. SE track: check A01 Broken Access Control right away. It is the most common hole in student prototypes and the easiest item to test — five minutes, and it often reveals a real Critical bug.